Container Technology

Docker & Kubernetes Administration & Security

Day 2

Kubernetes Architecture · Key Resources · Helm

Day 2 — Agenda

Part 1 — Kubernetes Architecture

- What is Kubernetes and why does it exist?
- Control plane components — apiserver, etcd, scheduler, controller-manager
- Node components — kubelet, kube-proxy, CRI, CNI
- The reconciliation loop

Part 2 — Key Kubernetes Resources

- Pods — the atomic unit
- Deployments, ReplicaSets, StatefulSets, DaemonSets
- Services — ClusterIP, NodePort, LoadBalancer, Headless
- Ingress and IngressClass
- ConfigMaps and Secrets
- PersistentVolumes and PersistentVolumeClaims
- Namespaces, ResourceQuotas, LimitRanges

Part 3 — Helm

- What is Helm and why use it?
- Chart structure — Chart.yaml, values.yaml, templates
- Managing repositories
- Installing, upgrading, rolling back
- Creating your own chart
- Hands-on: package and deploy an application

Part 1 — Kubernetes Architecture

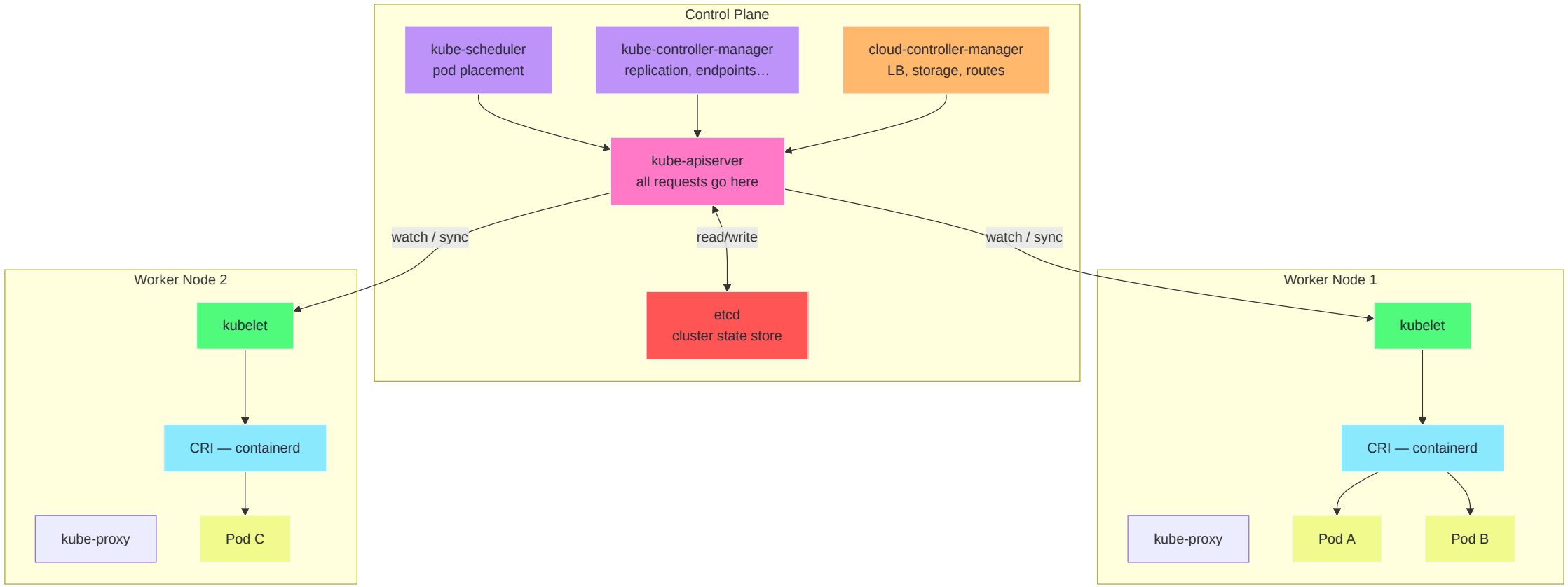
What is Kubernetes?

Kubernetes (K8s) is an open-source **container orchestration platform** originally developed at Google (Borg lineage), donated to the CNCF in 2014.

It solves the problems that arise when running containers **at scale**:

Problem	Kubernetes Solution
Containers die — restart them	Health checks + auto-restart
Need more instances under load	HorizontalPodAutoscaler
Route traffic to healthy pods	Services + readiness probes
Roll out new versions safely	Rolling deployments
Run containers on many nodes	Scheduler
Store app configuration	ConfigMaps + Secrets
Persistent storage for DBs	PersistentVolumes
Isolate teams and workloads	Namespaces + RBAC

Kubernetes Cluster Architecture



Control Plane — kube-apiserver

The **single gateway** to the entire cluster. Every read and write goes through it.

```
# The API server is just a REST API
curl -k https://localhost:6443/api/v1/namespaces \
  --cacert /etc/kubernetes/pki/ca.crt \
  --cert /etc/kubernetes/pki/admin.crt \
  --key /etc/kubernetes/pki/admin.key

# kubectl is a wrapper around these REST calls
kubectl get namespaces -v=6  # verbose — shows actual HTTP requests
```

Responsibilities:

- Authentication and authorisation of every request
- API versioning and conversion
- Admission control (mutation + validation)
- Watching etcd and notifying controllers
- Aggregating extension APIs (metrics-server, CRDs)

Control Plane — etcd

etcd is a distributed key-value store that holds the **entire cluster state** — every object, every resource definition.

```
# etcd stores keys under /registry/...
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/healthcheck-client.crt \
  --key=/etc/kubernetes/pki/etcd/healthcheck-client.key \
  get /registry/pods/default --prefix --keys-only

# Values are protobuf-encoded — decrypt with auger:
# github.com/jpbetz/auger
```

Security critical:

- etcd stores Secrets in **base64 only** by default — not encrypted
- Enable **encryption at rest** (`EncryptionConfiguration`) in production
- Restrict etcd access — only the API server should reach it

Control Plane — Scheduler and Controller Manager

kube-scheduler

Assigns pods to nodes. Considers:

- Resource requests vs. node capacity
- Node affinity / anti-affinity rules
- Taints and tolerations
- Pod topology spread constraints

kube-controller-manager

Runs a set of control loops. Each loop watches the API server and reconciles:

Controller	Watches	Acts on
Node controller	Node heartbeats	Marks unreachable nodes
Replication controller	ReplicaSet desired count	Creates/deletes pods
Endpoints controller	Services + Pods	Updates endpoint slices
Job controller	Job completions	Creates pods, marks done
Namespace controller	Namespaces	Cleans up deleted namespaces

The Reconciliation Loop

Every Kubernetes controller follows the same pattern:

```
1. WATCH — observe current state of world (from API server)
2. DIFF  — compare current state to desired state (the spec)
3. ACT   — take the minimum action to converge current → desired
4. REPEAT
```

```
// Pseudocode for a simple controller
for {
    desired := getDesiredState(apiserver)
    current := getCurrentState(apiserver)
    if desired != current {
        reconcile(current, desired)
    }
    time.Sleep(resyncPeriod)
}
```

> This is the fundamental philosophy of Kubernetes. **Declare what you want** — the system continuously drives toward that state. It is the same philosophy as Puppet, applied to infrastructure orchestration.

Node Components

kubelet

- Runs on every node
- Watches the API server for pods assigned to its node
- Starts containers via the CRI (Container Runtime Interface)
- Reports node and pod status back to the API server
- Runs health checks (liveness, readiness, startup probes)

kube-proxy

- Maintains iptables / IPVS / eBPF rules for Service routing
- When a Service is created, kube-proxy programs rules on every node so that ClusterIP → Pod IP forwarding works

Container Runtime (CRI)

- containerd or CRI-O receives pod specs from kubelet
- Creates namespaces, pulls images, starts containers
- Reports container status to kubelet

Part 2 — Key Kubernetes Resources

Pods — The Atomic Unit

A Pod is the smallest deployable unit in Kubernetes — it wraps one or more tightly-coupled containers that **share a network namespace and storage volumes**.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
  namespace: production
  labels:
    app: web
    version: "1.2.3"
spec:
  containers:
  - name: nginx
    image: nginx:1.27-alpine
    ports:
    - containerPort: 80
    resources:
      requests:        # scheduler uses this to place the pod
        cpu: "100m"
        memory: "128Mi"
      limits:          # cgroup ceiling — OOM kill if exceeded
        cpu: "500m"
        memory: "256Mi"
  restartPolicy: Always
```

Pod Probes — Health Signals

Kubernetes has three types of probes. Configuring them correctly is critical.

```yaml
spec:
  containers:
  - name: app
    livenessProbe:        # if this fails → restart the container
      httpGet:
        path: /healthz
        port: 8080
      initialDelaySeconds: 10
      periodSeconds: 5
      failureThreshold: 3

    readinessProbe:       # if this fails → remove from Service endpoints
      httpGet:
        path: /ready
        port: 8080
      periodSeconds: 5

    startupProbe:         # gives slow apps time to start before liveness kicks in
      httpGet:
        path: /healthz
        port: 8080
      failureThreshold: 30
      periodSeconds: 10
```

Deployments

A **Deployment** manages a ReplicaSet and enables declarative rolling updates and rollbacks.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # extra pods during update
      maxUnavailable: 0    # no downtime — keep all replicas up
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: ghcr.io/myorg/web:1.2.3
```

```
kubectl rollout status deployment/web
kubectl rollout history deployment/web
kubectl rollout undo deployment/web --to-revision=2
```

StatefulSets and DaemonSets

StatefulSet — for stateful workloads (databases, Kafka, Zookeeper)

- Pods get **stable network identities**: `pod-0`, `pod-1`, `pod-2`
- Pods start and stop **in order**
- Each pod gets its own **PersistentVolumeClaim** — data survives pod restarts
- Used for: PostgreSQL, Cassandra, Elasticsearch, etcd

DaemonSet — run exactly one pod per node

- Automatically adds a pod when a new node joins the cluster
- Automatically removes a pod when a node is drained
- Used for: log collectors (Fluentd), node monitors (node-exporter), CNI plugins, security agents

```
# Useful DaemonSet examples in a cluster
kubectl get daemonset -A
# NAMESPACE     NAME                DESIRED  CURRENT  READY
# kube-system   kube-proxy          3        3        3
# monitoring    node-exporter       3        3        3
# security      falco               3        3        3
```

Services — Stable Network Endpoints

Pods are ephemeral — their IPs change. A **Service** provides a stable virtual IP (ClusterIP) that routes to healthy pods via label selectors.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  selector:
    app: web            # routes to all pods with this label
  ports:
  - port: 80
    targetPort: 8080
  type: ClusterIP       # only reachable within the cluster
```

Type	Reachability	Use Case
`ClusterIP`	Cluster-internal only	Service-to-service
`NodePort`	Node IP + high port	Direct node access (dev/test)
`LoadBalancer`	Cloud load balancer IP	External traffic (prod)
`ExternalName`	CNAME to external DNS	External service aliasing
Headless (`clusterIP: None`)	DNS per pod	StatefulSet discovery

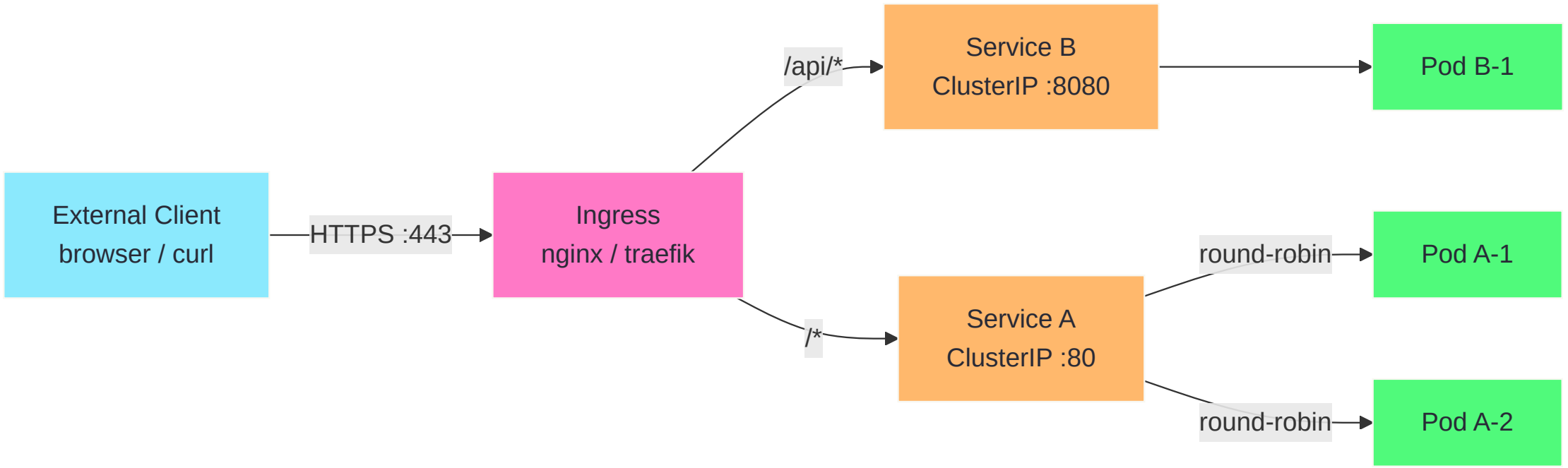
Ingress and IngressClass

An **Ingress** exposes HTTP/HTTPS routes from outside the cluster to Services inside it.

An **IngressClass** selects which Ingress controller handles it.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - app.example.com
    secretName: app-tls-cert
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-svc
            port:
              number: 8080
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-svc
            port:
              number: 80
```

Kubernetes Networking



ConfigMaps and Secrets

```yaml
# ConfigMap — non-sensitive configuration
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: info
  config.yaml: |
    server:
      port: 8080
      timeout: 30s

---
# Secret — sensitive values (base64 encoded — NOT encrypted!)
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  username: cG9zdGdyZXM=          # echo -n 'postgres' | base64
  password: c3VwZXJzZWNyZXQ=      # echo -n 'supersecret' | base64
```

> **Security warning:** Kubernetes Secrets are only base64 encoded by default — they are not encrypted. Enable **Encryption at Rest** (`EncryptionConfiguration`) or use an external secret manager (HashiCorp Vault, AWS Secrets Manager, External Secrets Operator).

Consuming ConfigMaps and Secrets in Pods

```yaml
spec:
  containers:
  - name: app
    env:
    # From ConfigMap
    - name: APP_ENV
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: APP_ENV

    # From Secret
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: password

    # Mount entire ConfigMap as files
    volumeMounts:
    - name: config
      mountPath: /etc/app/config.yaml
      subPath: config.yaml

  volumes:
  - name: config
    configMap:
      name: app-config
```

PersistentVolumes and PersistentVolumeClaims

```yaml
# PersistentVolumeClaim — what a pod requests
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-data
spec:
  accessModes:
    - ReadWriteOnce       # one pod can mount read-write
  storageClassName: gp3   # matches a StorageClass
  resources:
    requests:
      storage: 50Gi
```

Access Mode	Description	Use Case
`ReadWriteOnce` (RWO)	One node, read-write	Databases
`ReadOnlyMany` (ROX)	Many nodes, read-only	Config, static assets
`ReadWriteMany` (RWX)	Many nodes, read-write	Shared storage (NFS/EFS)

```
kubectl get pvc -A              # see all PVCs
kubectl describe pvc postgres-data   # check binding status
kubectl get pv                  # see the backing PersistentVolumes
```

Namespaces, ResourceQuotas, LimitRanges

```yaml
# Namespace — logical cluster partition
apiVersion: v1
kind: Namespace
metadata:
  name: team-alpha
  labels:
    pod-security.kubernetes.io/enforce: restricted

---
# ResourceQuota — total limits for the namespace
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-alpha-quota
  namespace: team-alpha
spec:
  hard:
    pods: "20"
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    count/secrets: "10"

---
# LimitRange — per-container defaults and caps
apiVersion: v1
kind: LimitRange
metadata:
  name: team-alpha-limits
  namespace: team-alpha
spec:
  limits:
  - type: Container
    default:
      cpu: "500m"
      memory: "256Mi"
    defaultRequest:
      cpu: "100m"
      memory: "128Mi"
    max:
      cpu: "2"
      memory: "2Gi"
```

Hands-on Preview — Day 2 Exercise 1

Deploy a full application stack:

1. Create a Namespace with a ResourceQuota
2. Deploy a backend API with a Deployment and readiness probe
3. Expose it with a ClusterIP Service
4. Deploy a PostgreSQL StatefulSet with a PVC
5. Wire credentials via a Secret
6. Expose the frontend via an Ingress

Goal: A running multi-tier application managed entirely by Kubernetes manifests.

See `exercises/day-2/exercise-1.md` for full lab instructions.

Part 3 — Helm

What is Helm?

Helm is the **package manager for Kubernetes** — it groups related Kubernetes manifests into a versioned, reusable package called a **chart**.

Without Helm:

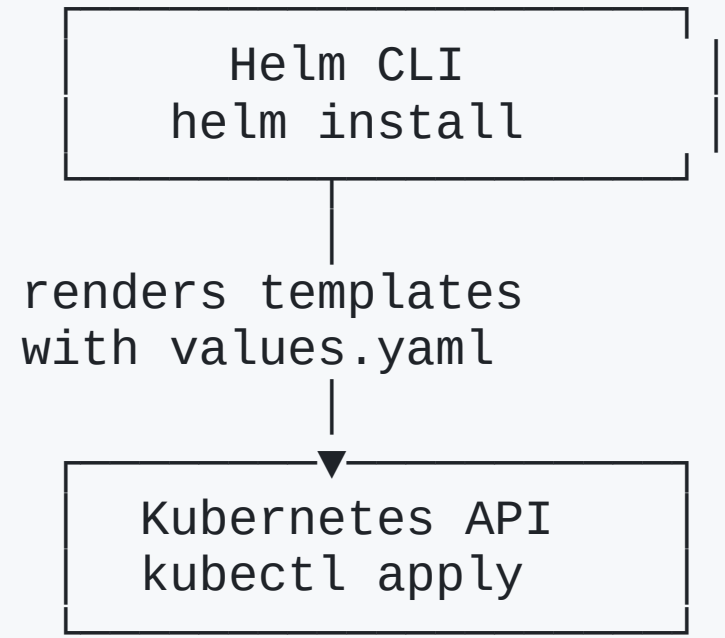
```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f ingress.yaml
kubectl apply -f configmap.yaml
# ... and repeat for every environment with manual edits
```

With Helm:

```
helm install myapp ./mychart --values prod-values.yaml
helm upgrade myapp ./mychart --values prod-values.yaml --set image.tag=1.2.4
helm rollback myapp 1
```

Helm tracks **releases** — each install/upgrade is a revision stored as a Secret in the cluster.

Helm Architecture




```
Release metadata stored as Secrets in the target namespace:
  sh.helm.release.v1.myapp.v1
  sh.helm.release.v1.myapp.v2
  ...
```

Chart Structure

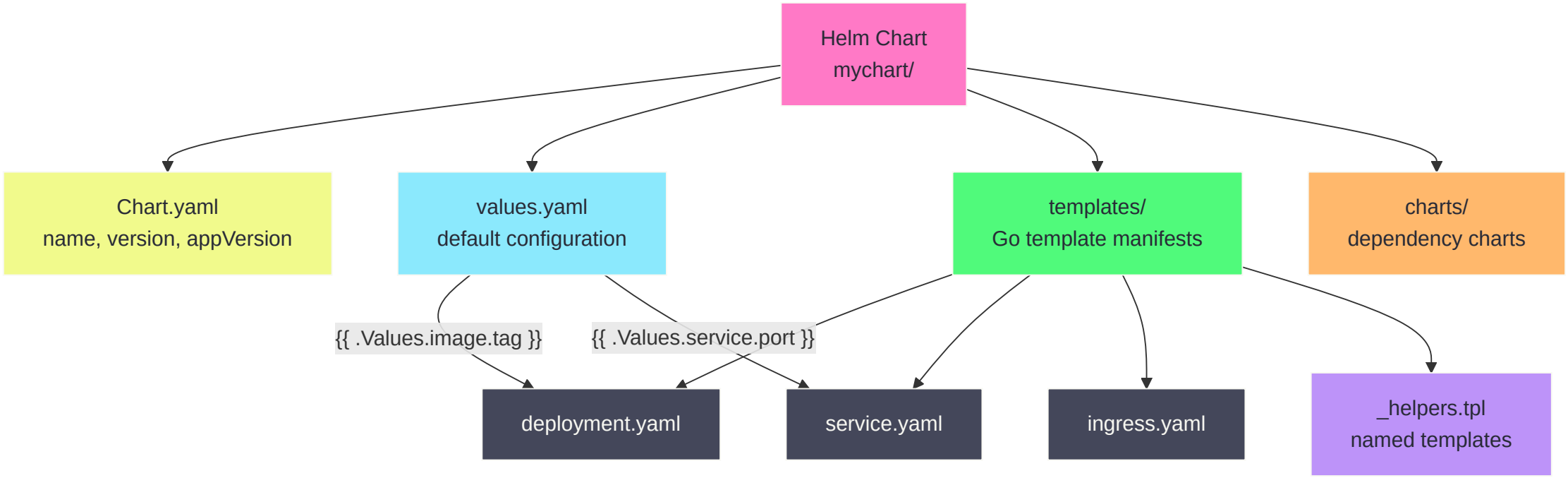


Chart.yaml and values.yaml

```yaml
# Chart.yaml — chart metadata
apiVersion: v2
name: myapp
description: My web application
type: application
version: 0.3.1           # chart version (SemVer)
appVersion: "1.2.3"      # app version (for display)
dependencies:
- name: postgresql
  version: "14.x.x"
  repository: https://charts.bitnami.com/bitnami
  condition: postgresql.enabled
```

```yaml
# values.yaml — default values (overridable)
replicaCount: 2

image:
  repository: ghcr.io/myorg/myapp
  tag: ""                # defaults to chart appVersion
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: false
  host: app.example.com

resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi
```

Templates

Templates use Go text/template syntax with Helm-specific functions.

```
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "myapp.fullname" . }}
  labels:
    {{- include "myapp.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "myapp.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "myapp.selectorLabels" . | nindent 8 }}
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
        imagePullPolicy: {{ .Values.image.pullPolicy }}
        ports:
        - containerPort: 8080
```

Managing Repositories

```
# Add repositories
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo add jetstack https://charts.jetstack.io

# Update all repos
helm repo update

# Search for charts
helm search repo postgresql
helm search hub nginx        # search ArtifactHub

# Show chart information
helm show chart bitnami/postgresql
helm show values bitnami/postgresql
```

Installing and Managing Releases

```
# Install with default values
helm install my-postgres bitnami/postgresql

# Install with custom values
helm install my-postgres bitnami/postgresql \
  --namespace database \
  --create-namespace \
  --values postgres-values.yaml \
  --set auth.postgresPassword=mysecret

# List all releases
helm list -A

# Check release status
helm status my-postgres -n database

# Upgrade (apply new chart version or values)
helm upgrade my-postgres bitnami/postgresql \
  --namespace database \
  --values postgres-values.yaml \
  --reuse-values

# Roll back to a previous revision
helm rollback my-postgres 2 -n database

# Uninstall (removes all K8s resources + release history)
helm uninstall my-postgres -n database
```

Creating Your Own Chart

```
# Scaffold a new chart
helm create myapp
# Creates: myapp/Chart.yaml, values.yaml, templates/...

# Lint for errors
helm lint ./myapp

# Render templates locally (dry-run)
helm template myapp ./myapp --values prod-values.yaml

# Package into a .tgz archive
helm package ./myapp

# Publish to OCI registry (modern approach)
helm push myapp-0.3.1.tgz oci://ghcr.io/myorg/charts

# Install from OCI registry
helm install myapp oci://ghcr.io/myorg/charts/myapp --version 0.3.1
```

Helm Best Practices

Practice	Why
Pin chart versions in CI	Reproducible deployments
Never store plain secrets in values	Use external-secrets or Vault
Use `helm diff` before upgrading	Catch unexpected changes
Tag releases with chart + app version	Traceability
Keep templates simple — logic in `_helpers.tpl`	Maintainability
Test with `helm test` hooks	Automated verification
Use `--atomic` in CI	Auto-rollback on failure

```
# helm diff (requires helm-diff plugin)
helm plugin install https://github.com/databus23/helm-diff
helm diff upgrade my-postgres bitnami/postgresql --values postgres-values.yaml

# Atomic install — rollback if any resource fails
helm upgrade --install myapp ./myapp --atomic --timeout 5m
```

Day 2 — Summary

Topic	Key Takeaway
K8s architecture	API server is the single source of truth; etcd stores all state
Reconciliation loop	Controllers continuously converge current state to desired state
etcd security	Enable encryption at rest — Secrets are not encrypted by default
Pods	Smallest unit; always set requests and limits
Deployments	Rolling updates, rollback, replica management
StatefulSets	Ordered pods, stable names, per-pod PVCs
Services	Stable VIP; type determines reachability
Ingress	HTTP routing + TLS termination at the cluster edge

Day 2 — Exercises

- **Exercise 1** — Deploy a multi-tier application with Deployments, Services, Ingress, StatefulSet, and Secrets
- **Exercise 2** — Package the application as a Helm chart and deploy it via `helm install`

See `exercises/day-2/` for full lab instructions.